

TimeSiam: Teaching Time-Series Models to Learn From Their Own Past

An ICML 2024 framework from Tsinghua University reframes self-supervised pre-training as reconstructing the present from the past, setting a new state of the art across 13 benchmarks.

Self-supervised pre-training reshaped how we build models for images and text: learn general-purpose representations from mountains of unlabeled data first, then fine-tune on the task you actually care about. Time series looks like the perfect next candidate. Industrial sensors, wearables, and the broader Internet of Things (IoT) generate a staggering, ever-growing stream of unlabeled measurements, and almost none of it is annotated. The temptation is obvious: borrow the recipes that worked for vision and language and point them at time series.

The trouble is that those recipes fit awkwardly. When you randomly mask points scattered across a series, you can shatter the smooth temporal structure the signal depends on, sometimes making reconstruction so hard the model learns very little. Contrastive learning has the opposite failure mode: comparing whole series for similarity tends to ignore the fine-grained correlations inside them, and it leans on data augmentations that are notoriously difficult to design for temporal data. Both families quietly overlook the one thing that makes a time series a time series: how a past observation is correlated with the present one.

TimeSiam, from a group at Tsinghua University and published at ICML 2024, takes that correlation as its central object. Instead of masking a window in isolation, it samples a past subseries and a current subseries from the same sequence and trains a Siamese network to reconstruct the masked present from the past. That single design choice forces the model to internalize how distant moments in a series relate, and it turns out to transfer remarkably well across tasks and domains.

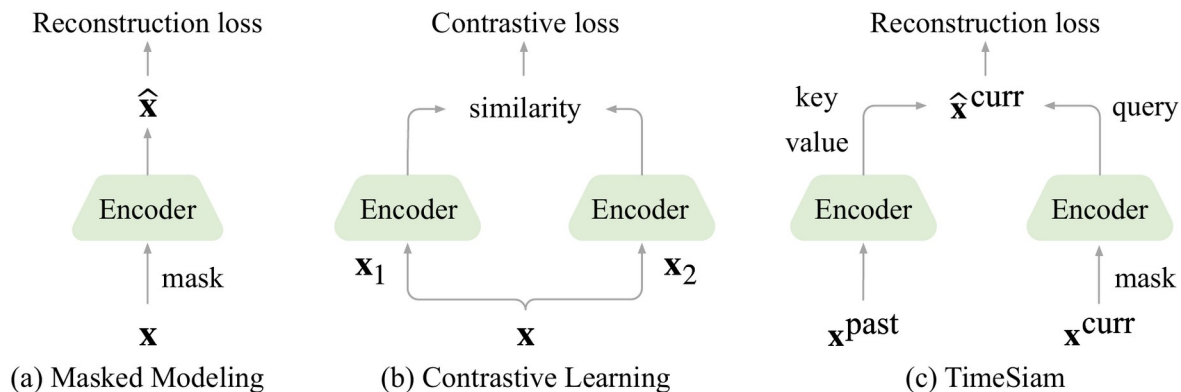


Figure 1. Three self-supervised recipes for time series. (a) Masked modeling reconstructs a masked series in isolation. (b) Contrastive learning pushes apart different series while pulling together two augmentations of the same series. (c) TimeSiam instead reconstructs a masked current window from a randomly sampled past window, making cross-time correlation the learning signal.

Paper: <https://arxiv.org/abs/2402.02475>

Code: <https://github.com/thuml/TimeSiam>

Why borrowed recipes fall short

It helps to be concrete about what goes wrong. Masked modeling was designed for tokens and image patches, where masking removes a discrete unit that the surrounding context can still infer. A time series is continuous and locally smooth, so masking scattered points either leaves an easy interpolation problem or, at high masking ratios, destroys enough structure that the task stops teaching anything useful. Contrastive methods sidestep reconstruction, but they hinge on augmentations that preserve semantics while changing appearance, and for temporal signals it is genuinely unclear which transformations do that. Series-wise similarity also collapses a whole window into one representation, throwing away the within-window dynamics that downstream forecasting and classification both rely on.

The motivation for a better recipe is not academic. The volume of unlabeled time series flowing off IoT sensors and wearable devices keeps growing, and a pre-training method that explicitly models temporal correlation could finally put that data to work for forecasting and classification without hand labeling. The question TimeSiam asks is simple: rather than treating each window on its own, why not build the pre-training task around the relationship between temporally distanced observations?

The core idea: reconstruct the present from the past

TimeSiam builds its pretext task from a pair the authors call Siamese subseries. From a single long series it randomly samples two windows: a past subseries and a current subseries. The current window is lightly corrupted with a simple augmentation such as masking. Two encoders with shared weights process the past and current windows, and a decoder with cross-attention reconstructs the masked current series, using the past window as the key and value and the corrupted present as the query. Training minimizes the squared error between the true and reconstructed current window. Because the only way to fill in the masked present is to exploit its correlation with the past, the encoder is pushed to learn exactly the temporal structure that masking and contrastive objectives leave on the table.

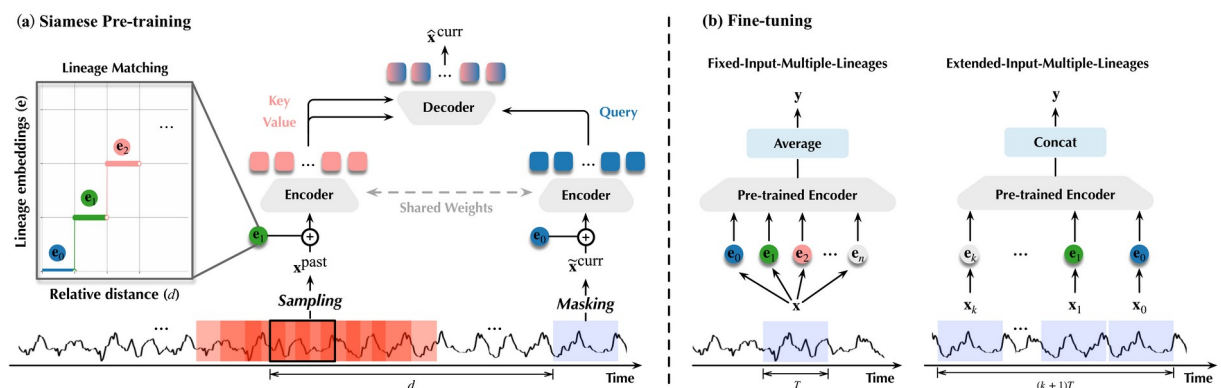


Figure 2. The TimeSiam framework. (a) During pre-training, a past window and a masked current window are drawn from the same series, encoded by shared-weight Siamese encoders, and a cross-attention decoder

reconstructs the current window from the past. A learnable lineage embedding, indexed by the relative distance between the two windows, is added before encoding. (b) At fine-tuning, multiple lineages are either averaged for a fixed-length input or concatenated to extend the input length.

Lineage embeddings: one model, many time gaps

A past window and a current window can sit close together in time or far apart, and the correlation between them changes with that gap. Forcing a single encoder to treat a one-step-back window and a thousand-step-back window identically would blur useful information. TimeSiam handles this with learnable lineage embeddings: a small set of vectors indexed by the relative distance between the past and current windows, added to the input representation before encoding. One model can then represent many temporal distances at once, and the final representation is an ensemble over these lineages. At fine-tuning, the lineage embeddings are averaged when the input length is fixed, or concatenated segment by segment when the model is adapted to a longer input, so a single model can stretch to inputs far longer than the windows it ever encountered while pre-training on the source data.

Does it work? Results across 13 benchmarks

The evaluation is deliberately broad, spanning 13 benchmarks and two mainstream tasks. Forecasting covers the four ETT subsets along with Weather, Electricity, Traffic, and Exchange; classification covers the EEG datasets AD and TDBrain and the ECG dataset PTB. On top of these established benchmarks the authors build two new large-scale, multi-domain pre-training corpora, TSLD-500M and TSLD-1G, where the larger one packs 13.9M examples drawn from diverse, non-overlapping domains and lets the paper stress-test cross-domain transfer.

The headline numbers are consistent and strong. On in-domain forecasting, TimeSiam lowers average mean squared error by 5.7% with a PatchTST backbone and 2.5% with iTransformer, relative to training from random initialization, and remember these backbones already forecast well from scratch. On in-domain classification it raises average accuracy by 11.5%. Across both settings it outperforms 8 strong self-supervised baselines, including reconstruction-based methods like SimMTM and contrastive ones like TS2Vec and CoST. The scatter plots below make the pattern easy to see: TimeSiam (the red star) sits in the top-left corner in both the in-domain and cross-domain panels, meaning the lowest forecasting error and the highest classification accuracy at once.

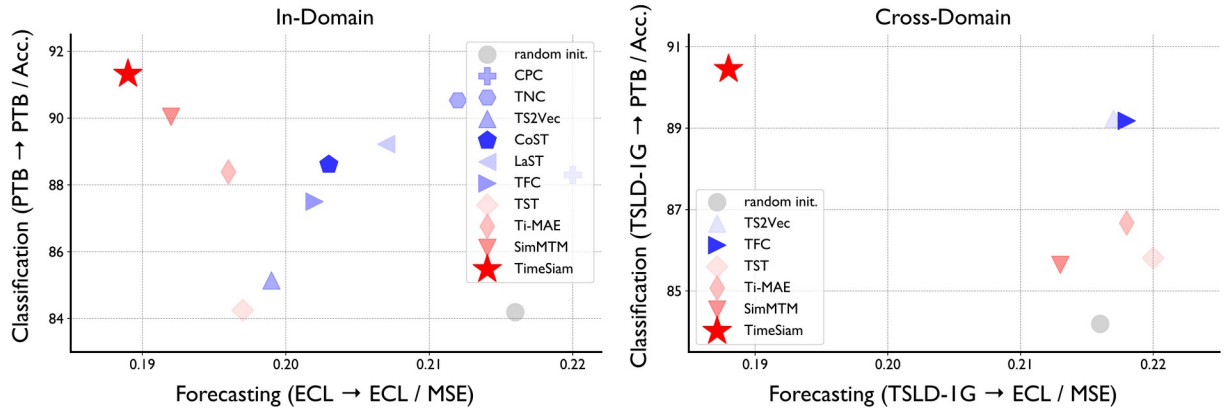


Figure 3. Forecasting error (MSE, lower is better) versus classification accuracy (higher is better) for many self-supervised baselines, in the in-domain setting (left) and the cross-domain setting where pre-training uses TSLD-1G (right). TimeSiam, the red star, is alone in the top-left of both panels, dominating both axes at once.

Table 1. In-domain gains of TimeSiam over random initialization, averaged across benchmarks.

Setting	Metric	TimeSiam vs. random init.
In-domain forecasting (PatchTST)	Avg MSE	5.7% lower
In-domain forecasting (iTransformer)	Avg MSE	2.5% lower
In-domain classification	Avg accuracy	11.5% higher

The cross-domain story is the most striking. Pre-training on the large, diverse TSLD-1G corpus and fine-tuning on an entirely different target dataset sometimes beats even in-domain pre-training, and TimeSiam remains the best method under the distribution shift that cross-domain transfer imposes. That is a useful signal for practitioners: the scale and diversity of the pre-training data genuinely pay off, and a model pre-trained once on broad, varied data can then be reused across many different downstream sources.

Adapting to longer inputs

One practical worry with any forecasting model is what happens when you feed it a longer history at fine-tuning time than it was trained on. Longer inputs carry more information but also more noise, and a standard model can actually get worse as the input grows. Because TimeSiam can concatenate lineage embeddings across segments, it adapts to extended inputs natively rather than degrading. The panels below track average error as the input length grows for four datasets, and the pattern is clear: the from-scratch baseline flattens or bends upward while TimeSiam keeps improving.

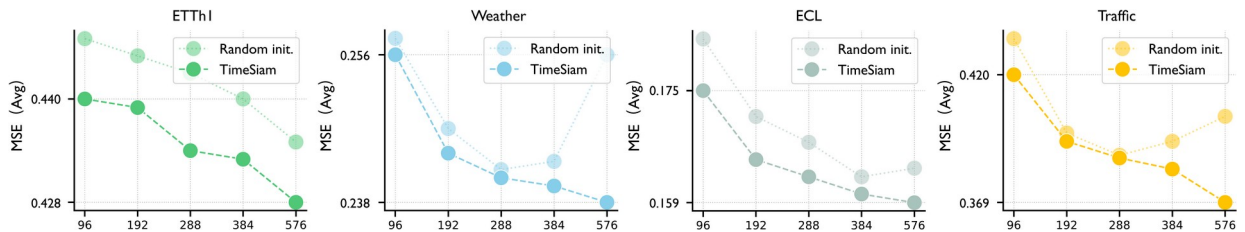


Figure 4. Fine-tuning to extended input lengths (96, 192, 288, 384, 576) on an iTransformer backbone, reporting MSE averaged over the forecast horizons 96, 192, 336, 720. As the input grows, training from random initialization can degrade, while TimeSiam continues to lower error.

What the ablations confirm

Ablations on the Traffic benchmark pin down which design choices matter. Reconstructing the current window from a past one clearly beats plain self-reconstruction, which validates the core Siamese idea rather than the masking alone. The masking ratio has a sweet spot near 25%: masking only 15% makes the pretext task too easy to teach anything, while masking 75% makes it too hard to guide useful learning. And lineage embeddings deliver consistent gains over random initialization; adding more of them keeps improving results up to a point, confirming that explicitly modeling several temporal distances is worth the small extra cost.

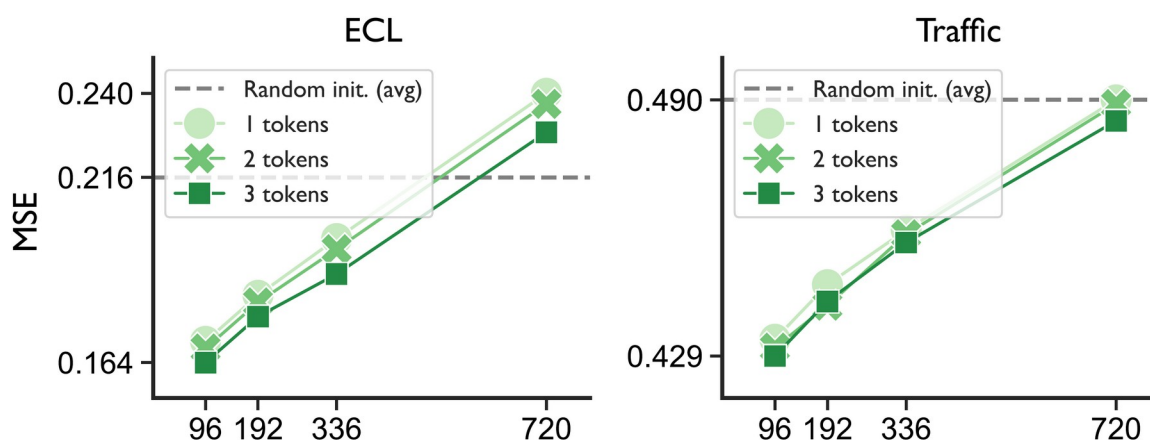


Figure 6. Effect of the number of lineage embeddings on ECL and Traffic, using 1, 2, and 3 lineage tokens under the input-96 in-domain setting. More lineage tokens push the curves below the random-initialization line across forecast horizons, showing that modeling multiple temporal distances helps.

The takeaway

TimeSiam's contribution is a reframing more than a new architecture. By posing time-series pre-training as past-to-current reconstruction between Siamese subseries, and equipping one model with learnable lineage embeddings to span different temporal distances, it captures exactly the correlations that masking and contrastive objectives miss. The framework is backbone-agnostic, dropping cleanly onto PatchTST, iTransformer, and TCN, and its benefits grow with larger and more diverse pre-training data. The honest boundary is that cross-domain pre-training does not always beat in-domain pre-training, since the gap between pre-training and fine-tuning domains still matters. But as a simple, general way to turn unlabeled time series into strong transferable representations, TimeSiam sets a clear new bar for both forecasting and classification. For practitioners the recipe is easy to adopt: pre-train once on whatever unlabeled series you already collect, attach the encoder to a standard forecaster or classifier, and fine-tune as usual. The broader lesson is that the design of the pretext task can matter more than raw scale, and for time series the task worth learning is how the past shapes the present.